

Repair Without Stopping: A Runtime for Online Evolution of Hybrid Code-VLA Robot Programs (Student Abstract)

Chukwuemeka Samuel Udoh

Ashesi University, 1 University Avenue, Berekuso, Ghana
+234 707 209 9407, udoh.emeka@ashesi.edu.gh, <https://udohchuks.github.io/>

Abstract

Coding agents can write and revise robot programs, and vision-language-action (VLA) models handle the contact-rich steps that code does poorly. In current hybrid programs, fixing a skill during deployment means restarting the task. We present a runtime that lets an agent repair skills while the task runs. A Rust loop plays robot actions at a fixed rate. A Python core hosts the skills, which can be replaced live and rolled back if they fail, using the revertible effects of Cordis. In simulation, with a scripted placeholder for the VLA, the control loop kept its timing while Python stalled for up to 2.7 s. The runtime rejected or rolled back all 80 broken repairs without a restart, and an LLM’s first repair was kept in 25 of 25 sessions. On a physics-simulated SO-101 arm, a halt enforced in Rust prevented contact with a bowl in 10 of 10 runs; without it, the arm hit the bowl every time.

Introduction

Coding agents write robot programs as readable skills that keep state and can be fixed one at a time (Liang et al. 2023; Fu et al. 2026). Code skills struggle in contact-rich steps, where VLA models do better (Shukor et al. 2025), and hybrid systems such as SkipVLA (Agrawal et al. 2026) combine both. Once such a program is deployed, however, fixing a skill usually means restarting it. Dynamic software updating can replace code while it runs, in general software and in robots (Hayden et al. 2012; El Yaacoub et al. 2023). A robot also needs a bad replacement to be rolled back, and the commands it has already sent to the arm to be cancelled. We study this combination as an online improvement loop. While the task runs, each repair is tried, then kept or rolled back, so the program improves without a human in the loop. “Without stopping” means the runtime is never restarted; the arm may still hold safely. We target robots that run the VLA on their own computer, where a restart means reloading model weights. Figures, tables and extra results are in the supplement.

Runtime Design

Live loop (Rust). Python cannot hold a fixed rate. VLA inference or agent calls can take seconds, and one call that holds Python’s global lock freezes every Python thread. A separate Rust process (Fig. 2) therefore runs a 30 Hz loop;

each step is a tick. Commands arrive as chunks, short lists of up to 50 target poses, stamped with the tick at which the first should play. Rust plays one action per tick and skips actions whose tick has passed. A new chunk replaces what is left of the old one. Rust rejects non-finite values and caps speed and acceleration. If Python’s heartbeat stops for 200 ms, Rust plays at most 10 more actions, then holds.

Skills and ownership (Python). The task is a behavior tree (Colledanchise and Ögren 2018) stepped at 10 Hz. Each leaf is a node, either a code skill or the VLA, and drives the arm by sending chunks. One node drives at a time. Each time a node starts driving, it gets a new ID (a token) stamped on its chunks. To stop a node, Python asks Rust to halt its token. Rust drops every waiting or playing chunk with that token or an older one, then brings the arm to rest. The runtime requests the halt when a node is replaced, throws an error, or one call runs over 0.5 s.

Repairs that can be undone. Skills are plugins. As in Cordis (Shi, Zhang, and Cui 2026), each step a plugin takes while loading comes with a matching undo. If loading crashes, the undos run newest-first. A repair (new skill version) loads beside the old one. It takes over only when the old version is idle, with no chunks waiting or playing. It is kept only if the next 3 episodes succeed, which we call its trial; otherwise the old version comes back.

Preliminary Validation

Kinematic tests. We used two 2-core machines without GPU, a cloud VM and a laptop VM, and a simple pick-and-place simulator (hand, cube, bowl). We stalled Python with VLA delays up to 2 s, a call that held the global lock for up to 2.7 s, a skill that throws errors, and killing Python. Rust kept sending commands in every condition; on the cloud VM, 20 of 34,362 ticks ran over one period late. Inline and threaded Python-only loops froze for up to 2.3 s. A hand-written fix for a buggy skill took over in 198 ms. A restart with SmolVLA loaded took about 20 s. We loaded 10 kinds of faulty repair, 10 times each; all 80 broken ones were rejected or rolled back, and Rust never played an action from a halted token. With a plain hot reload, six broken kinds stayed live and none of their 90 later episodes succeeded.

Repairs written by an LLM. We planted one of five simple bugs in the approach skill, such as a flipped sign. After two failed episodes, DeepSeek-V4.1-Flash received the

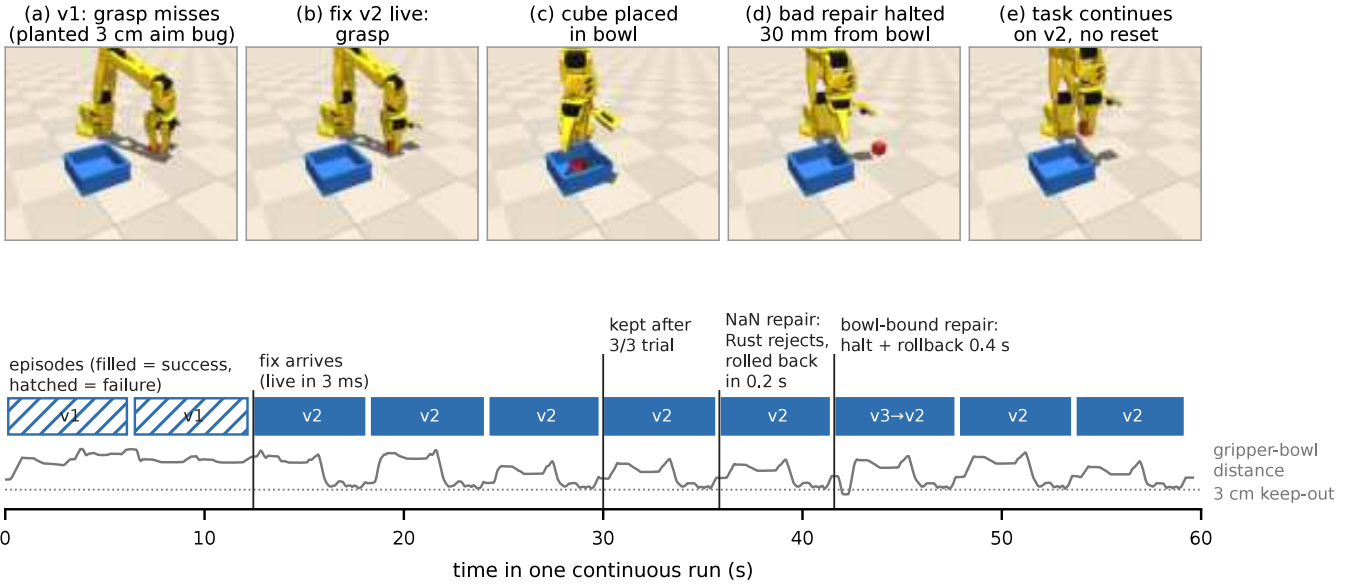


Figure 1: One continuous 60 s run on the MuJoCo SO-101 arm. Top: frames. Bottom: episodes (filled = success, hatched = failure) and repair events; v3→v2 = bad repair removed mid-episode; dotted line = 3 cm safety zone, active only during approach. The robot is never reset.

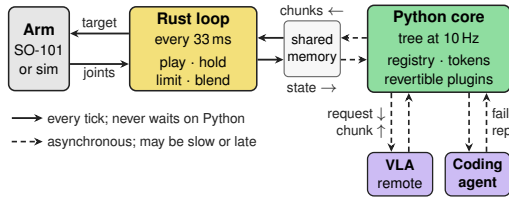


Figure 2: Runtime architecture.

skill’s code, the plugin API and a report of observed poses. In 25/25 sessions its first answer was kept, 22–66 s after the report, while episodes kept running. So the improvement loop can run without a human, at least for simple bugs.

Physics simulation. We ran the same runtime on a MuJoCo SO-101 arm, with the cube held by friction alone. In one continuous run (Fig. 1), the fix (v2) replaced the buggy v1 and was kept. We then injected two faulty repairs. Rust rejected the first, which sent invalid numbers, before any of its actions played. The second (v3) drove toward the bowl wall. A 3 cm safety check halted it through Rust, without contact. Both were removed, and all 8 episodes after the fix succeeded. When the arm was driven straight at the bowl, a Rust halt stopped it in 10/10 runs without contact. With no halt, or a halt only in Python, it hit the bowl in 10/10. So a halt must drop the commands already sent, in the layer that plays them.

Limitations and Next Steps

Simulation and a scripted VLA test the mechanisms, not task reliability. The bugs were planted, and the safety check reads exact distances from the simulator; on a real robot these must

come from perception. A real VLA also adds slow, variable inference on the same computer. Our next step is to deploy the runtime on a real SO-101 arm.

References

- Agrawal, K.; Rahman, M. A.; Yeh, R. A.; and Kingston, Z. 2026. SkipVLA: Skipping VLA Steps with Classical Planning for Fast Robot Manipulation. *arXiv preprint arXiv:2609.20648*.
- Colledanchise, M.; and Ögren, P. 2018. *Behavior Trees in Robotics and AI: An Introduction*. CRC Press.
- El Yaacoub, A.; Mottola, L.; Voigt, T.; and Rümmer, P. 2023. Scheduling Dynamic Software Updates in Mobile Robots. *ACM Transactions on Embedded Computing Systems*, 22(6).
- Fu, L.; Yu, J.; El-Refai, K.; Kou, E.; et al. 2026. CaP-X: A Framework for Benchmarking and Improving Coding Agents for Robot Manipulation. *arXiv preprint arXiv:2603.22435*.
- Hayden, C. M.; Smith, E. K.; Denchev, M.; Hicks, M.; and Foster, J. S. 2012. Kitsune: Efficient, General-Purpose Dynamic Software Updating for C. In *OOPSLA*.
- Liang, J.; Huang, W.; Xia, F.; Xu, P.; et al. 2023. Code as Policies: Language Model Programs for Embodied Control. In *IEEE International Conference on Robotics and Automation (ICRA)*.
- Shi, Y.; Zhang, W.; and Cui, T. 2026. A Programming Paradigm for Spatiotemporal Composability. *arXiv preprint arXiv:2608.25512*.
- Shukor, M.; Aubakirova, D.; Capuano, F.; Kooijmans, P.; et al. 2025. SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics. *arXiv preprint arXiv:2506.01844*.